
EMERITUS
Learn. From the world's best.

Linear Regression Workshop

EMERITUS
Learn. From the world's best.

Key Topics in the Async Session

A Quick Guide to Core Machine Learning Concepts

Supervised Learning and Linear Models

Predict outcomes from labelled data. Understand input-output relationships

Parameter Estimation

Learn how models find best-fit values using methods like Least Squares

Overfitting and Regularisation

Avoid overly complex models. Use regularisation to improve generalisation

Regularisation Techniques

Regularisation balances model accuracy with complexity

Simple Linear Regression Steps

Build, View, and Test a Basic Prediction Model

Train the Model

Use house size to predict price

Plot the Line

Show how the variables are related

Test Accuracy

Compare results on training and test data

Dataset Overview

Initial Steps to Prepare and Understand the Data

Import Libraries
Use pandas, numpy, matplotliband sklearn

Load Dataset
Load CSV into a DataFrame

Explore Data
View top rows to check format and content

Index	Area	Bedrooms	Bathrooms	Material	Locality	Price
0	1790	2	2	Concrete	Riverside	114300
1	2030	4	2	Concrete	Riverside	114200
2	1740	3	2	Concrete	Riverside	114800
3	1980	3	2	Concrete	Riverside	94700
4	2130	3	3	Concrete	Riverside	119800

Linear Regression Setup

Step 1: Import Required Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score
```

Linear Regression Setup

Step 2: Load the Dataset

```
df = pd.read_csv('house.csv')
df.head()
```

Linear Regression Setup

Step 3: Choose Feature and Target

- Use ‘Area’ as the input
- Predict ‘Price’ as the output

```
X = df[['Area']]
y = df['Price']
```

Linear Regression Setup

Step 4: Split the Data

- Divide data into training and testing sets
- 25% of data used for testing

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)
```

Linear Regression Setup

Step 5: Train the Model

```
model = LinearRegression()
model.fit(X_train, y_train)
```

Linear Regression Setup

Step 6: Plot Regression Line on Training Data

```
plt.scatter(X_train, y_train, color='skyblue', label='Train Data')
plt.plot(X_train, model.predict(X_train), color='red', label='Regression Line')
plt.xlabel('Area (sq ft)')
plt.ylabel('Price')
plt.title('Linear Regression – Training Set')
plt.legend()
plt.grid(True)
plt.show()
```

Linear Regression Setup

Step 7: Assess Accuracy on Train and Test Sets

```
# On Training Data


train_preds = model.predict(X_train)
print("Train R² Score:", r2_score(y_train, train_preds))
print("Train RMSE:", np.sqrt(mean_squared_error(y_train, train_preds)))
# On Test Data
test_preds = model.predict(X_test)
print("Test R² Score:", r2_score(y_test, test_preds))
print("Test RMSE:", np.sqrt(mean_squared_error(y_test, test_preds)))
```

Linear Regression Setup

Step 8: Plot Regression Line on Test Data

```
plt.scatter(X_test, y_test, colour='green', label='Test Data')
plt.plot(X_test, test_preds, colour='orange', label='Regression Line')
plt.xlabel('Area (sq ft)')
plt.ylabel('Price')
plt.title('Linear Regression – Test Set')
plt.legend()
plt.grid(True)
plt.show()
```

Make Predictions

Use Model on Test Data

- Predict house prices using test inputs

```
y_pred = model.predict(X_test)
```

Compare Predictions

Actual vs Predicted Prices (Scatter Plot)

```
plt.scatter(y_test, y_pred, colour='purple')
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], colour='red',
linestyle='--')  # Ideal line
plt.xlabel('Actual Price')
plt.ylabel('Predicted Price')
plt.title('Actual vs Predicted Prices')
plt.grid(True)
plt.show()
```

Model Summary

R^2 Score, Coefficient and Intercept

R^2 Score

```
r2 = r2_score(y_test, y_pred)
print(f"R² Score on Test Data: {r2:.2f}")
```

Coefficient and Intercept

```
print(f"Model Coefficient (slope): {model.coef_[0]:.2f}")
print(f"Model Intercept: {model.intercept_:.2f}")
```

Why Use Polynomial Regression?

When Linear Models Fall Short

Non-linear Patterns

When data relationships aren't straight lines

Visual Comparison

Straight vs Curved fit

Real-life Example

Age vs Income relationship

Multiple Linear Regression

Multiple Linear Regression

Predicting House Prices

- Apply multiple linear regression to housing data
- Convert categories to dummy variables
- Fit the model to the data
- Evaluate using R^2 and RMSE on train and test sets

Multiple Linear Regression Guide

Required Libraries

```
Import pandas as pd
Import numpy as np
Import matplotlib.pyplot as plt
From sklearn, import LinearRegression
From sklearn, use train_test_split
From sklearn.metrics, import mean_squared_error, r2_score
```

Multiple Linear Regression

Step 2 and 3

Step 2: Load Dataset

```
df = pd.read_csv("house.csv")
df.head()
```

Step 2: Handle Categorical Variables

- Convert categorical data with one-hot encoding

```
pd.get_dummies()
df_encoded = pd.get_dummies(df, ['Material', 'Locality'], drop_first=True,
dtype=int)
df_encoded.he
```

Multiple Linear Regression

Step 4: Define Features and Target

We'll use all columns except ‘Price’ as predictors.

```
X = df_encoded.drop('Price', 1)
y = df_encoded['Price']
```

Multiple Linear Regression: Model Training

Step 5 and Step 6

Step 5: Split Data into Train and Test

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)
```

Step 6: Train the Linear Regression Model

```
model = LinearRegression()
model.fit(X_train, y_train)
```

Multiple Linear Regression: Model Evaluation

Step 7: Assess Performance on Training and Test Data

Train Performance

```
y_train_pred = model.predict(X_train)
print("Train R²:", r2_score(y_train, y_train_pred))
print("Train RMSE:", np.sqrt(mean_squared_error(y_train, y_train_pred)))
```

Test Performance

```
y_test_pred = model.predict(X_test)
print("Test R²:", r2_score(y_test, y_test_pred))
print("Test RMSE:", np.sqrt(mean_squared_error(y_test, y_test_pred)))
```

Multiple Linear Regression: Visualise Predictions

Step 8: Actual vs Predicted (Test Data)

```
plt.scatter(y_test, y_test_pred, color='green')
plt.plot([y.min(), y.max()], [y.min(), y.max()], color='red',
linestyle='--')
plt.xlabel('Actual Price')
plt.ylabel('Predicted Price')
plt.title('Actual vs Predicted Prices (Test Set)')
plt.grid(True)
plt.show()
```

Polynomial Regression

Polynomial Regression

Modelling Non-Linear Relationships

Polynomial Regression models the relationship between variables using an n^{th} degree polynomial

Mathematical Form

$$y = \beta_0 + \beta_{1^x} + \beta_2 x^2 + \cdots + \beta_n x^n + \varepsilon$$

Key Characteristics:

- Extends linear regression with polynomial terms
- Linear in coefficients, solved like linear regression
- Captures non-linear data patterns

Why Use Polynomial Regression?

When to Use

- Data shows a curved or non-linear trend
- Linear regression underfits the data
- Need to capture feature interactions or higher-order effects

Advantages

- Easy to implement and interpret
- More flexible than linear regression for complex patterns

Considerations

- Risk of overfitting with high-degree polynomials, especially on small datasets
- May require feature scaling for stability
- Interpretability decreases as polynomial degree increases

Polynomial Regression

Using PolynomialFeatures for Improved Model Fit

Extends Linear Regression with polynomial terms

Remains linear in coefficients, solved like linear regression

Captures non-linear patterns in data

Polynomial Regression

Load required libraries

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
import matplotlib.pyplot as plt
```

Polynomial Regression

```
#Create polynomial features
poly = PolynomialFeatures(degree=2, include_bias=False)
X_train_poly = poly.fit_transform(X_train)
X_test_poly = poly.transform(X_test)

# Train the model
model = LinearRegression()
model.fit(X_train_poly, y_train)
```

Model Evaluation – Polynomial Regression

Check Accuracy on Training and Test Sets

```
# Predict on train and test
y_train_pred = model.predict(X_train_poly)
y_test_pred = model.predict(X_test_poly)

# Evaluate model
train_rmse = np.sqrt(mean_squared_error(y_train, y_train_pred))
test_rmse = np.sqrt(mean_squared_error(y_test, y_test_pred))
train_r2 = r2_score(y_train, y_train_pred)
test_r2 = r2_score(y_test, y_test_pred)
```

Model Evaluation – Polynomial Regression

Check Accuracy on Training and Test Sets

```
# Prepare results
results = {
    "Train RMSE": train_rmse,
    "Test RMSE": test_rmse,
    "Train R2": train_r2,
    "Test R2": test_r2
}

results
```

Q&A Session

- Open floor for questions and clarifications.